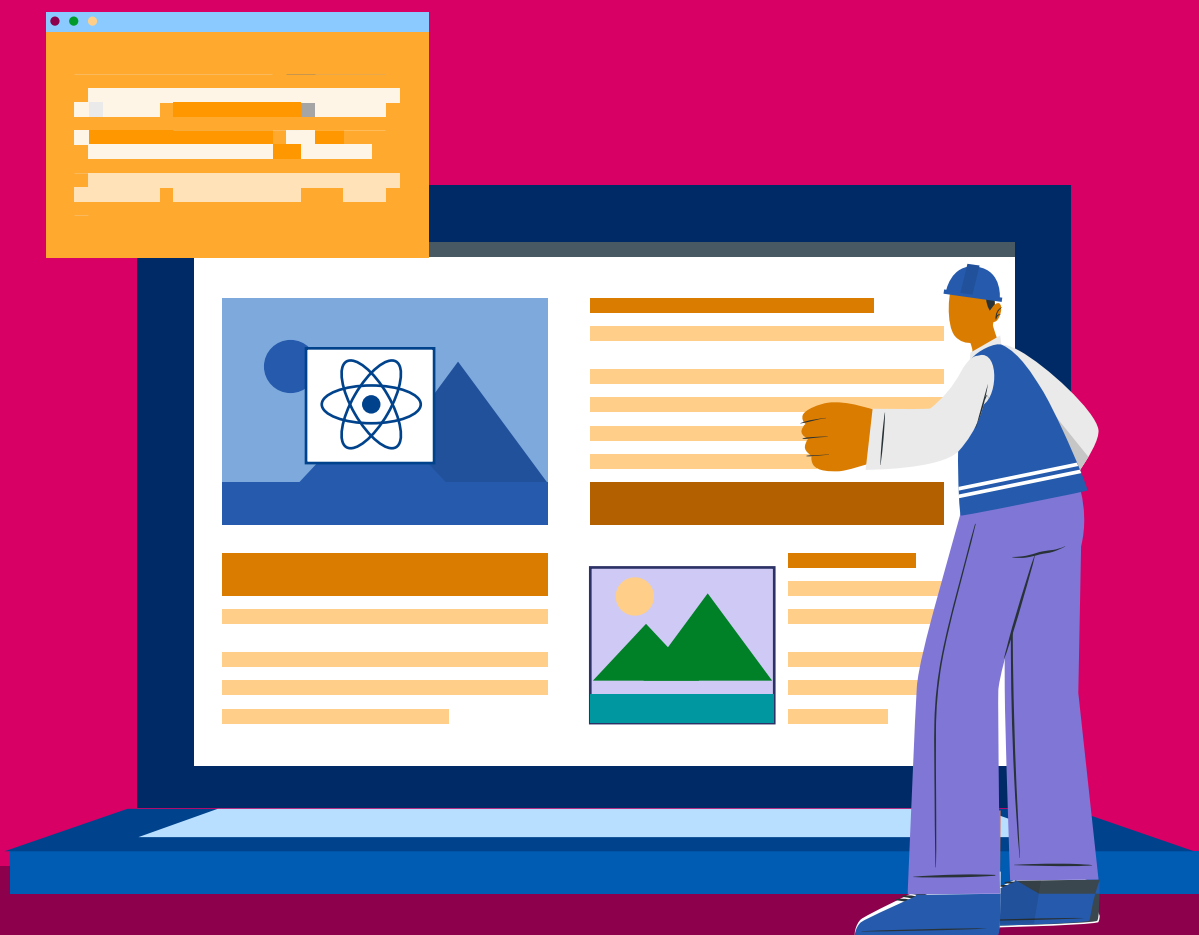


REACT



India's best tech learning company

Learn industry-relevant skills with top tech veterans



126% Avg. CTC Hike

Top 1% Industry Instructors

900+ Placement Partners

Programs We Offer

Software Development
Course



Data Science &
Machine Learning



The Scaler Recipe to Transform Your Career

A structured & flexible
program, that cares for you

Be Mentored 1:1 by
Experienced Professionals

Become part of a thriving
community for life

Discover & connect with Alumni



Sudhanshu Gera
Software Engineer III

Pre Scaler
Wipro Limited

Post Scaler
Walmart*

↗ 200% Hike



Ankit Pangasa
Senior Software Engineer

Pre Scaler
Adobe

Post Scaler
Google

↗ 200% Hike

Connect with Alumni

JavaScript library for building user interfaces.



Features

○ Component-based architecture

React applications are built as a tree of reusable and composable components, simplifying development.

○ Virtual DOM

Optimizes rendering by updating only the changed parts of the actual DOM, leading to improved performance.

○ Unidirectional data flow

Easier to trace and understand how data changes propagate through a component tree.

○ Declarative Syntax

Easier to understand and debug code by describing the desired outcome rather than the steps to achieve it.

○ JSX (JavaScript XML)

Syntax extension for JavaScript that allows embedding HTML-like code within JavaScript

○ Reusable Components

Promotes the creation of reusable components, fostering modular and maintainable codebases.

○ One-Way Data Binding

Reducing unexpected side effects and making it easier to reason about the application's state.

02

Types of Components

Functional Components

```
import React from 'react';
const FunctionalComponent = (props) => {
  return (
    <div>
      {/* JSX content */}
    </div>
  );
};
```

Basic structure using arrow functions

Class Components

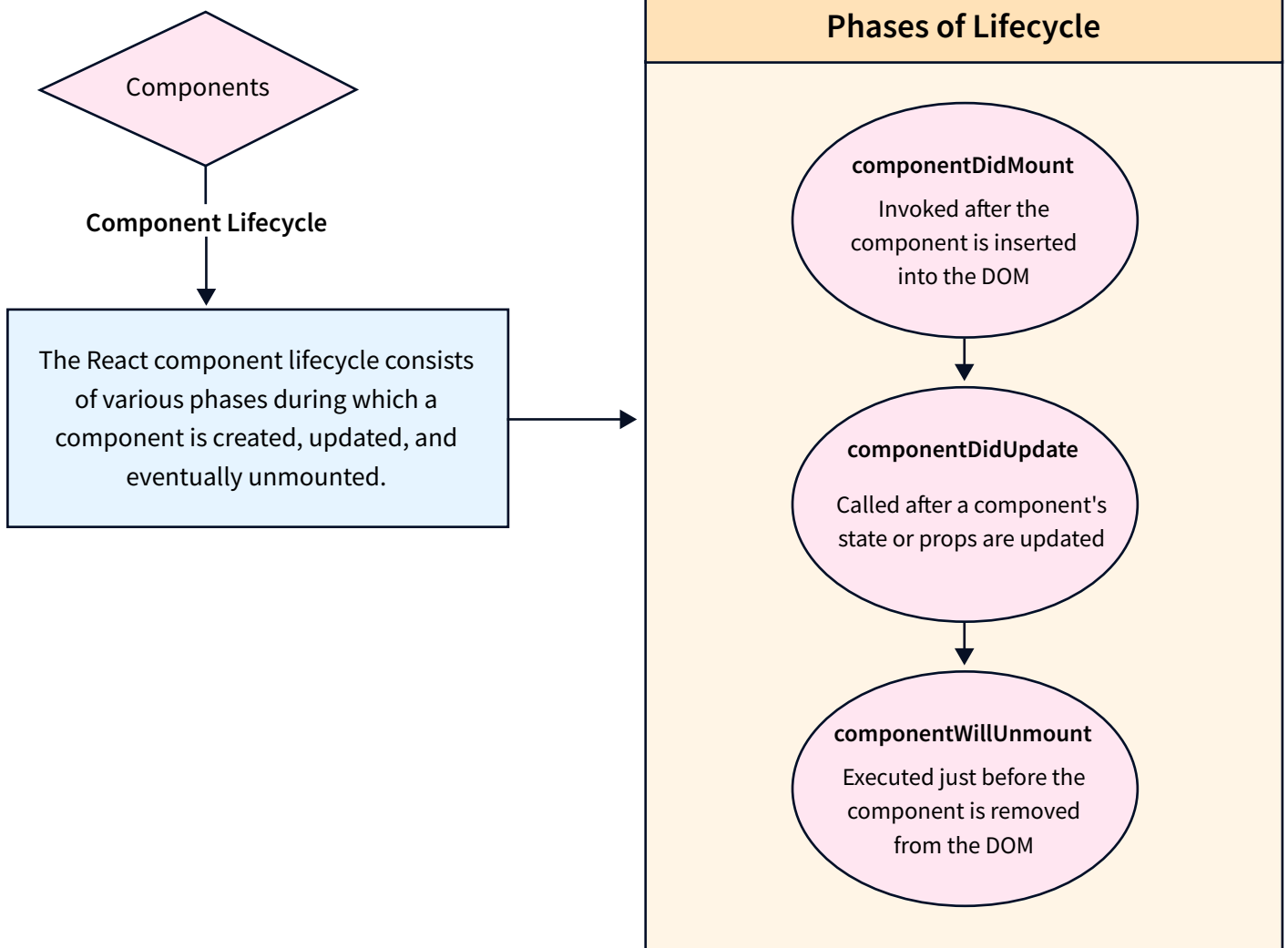
```
import React, { Component } from 'react';
class ClassComponent extends Component {
  constructor(props) {
    super(props);
    // Component state initialization
  }
  render() {
    return (
      <div>
        {/* JSX content */}
      </div>
    );
  }
}
```

Class-based component syntax

React Fragments

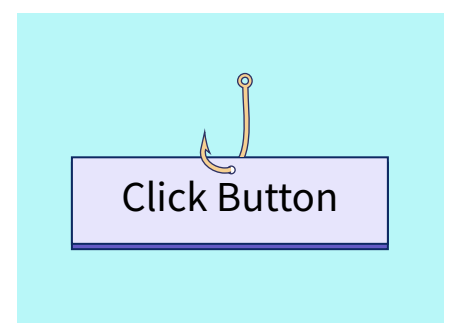
```
import React from 'react';
const ComponentWithFragment = () => {
  return (
    <>
      <h1>React Fragments</h1>
      <p>Group multiple elements.</p>
    </>
  );
};
```

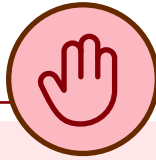
Short syntax for returning multiple elements



03 Hooks

Functions that enable functional components to have state and lifecycle features previously available only in class components.





Rules of Hooks

- **Top-Level Only:** Called only at the top level of a functional component or a custom hook.
- **Consistent Invocation:** Hooks must be invoked in the same order on every render.
- **No Conditional Hooks:** Do not call hooks conditionally. Hooks should always be called in the same order.
- **Inside Functional Components:** Should be used only in functional components and custom hooks.
- **Use in Custom Hooks:** Must start with the word "use" to indicate that they are hooks.
- **One Level of Abstraction:** Each custom hook should represent one level of abstraction.

Hooks in React

useState Hook	useCallback Hook
<pre>import React, { useState } from 'react'; const ButtonComponent = () => { const [state, setState] = useState("On"); return (<div> <button onClick={() => setState("Off")} >{state}</button> </div>); };</pre> Managing state in functional components	<pre>import React, { useState, useCallback } from 'react'; const MemoizedCallback = () => { const [count, setCount] = useState(0); const handleClick = useCallback(() => { setCount(count + 1); }, [count]); return <button onClick={handleClick}>Increment</button>; };</pre> Memoizing functions for optimization

useEffect Hook

```
import React, { useEffect } from 'react';
const MyComponent = () => {
  useEffect(() => {
    // Effect code (side effects, data fetching, etc.)
    return () => {
      // Cleanup code (optional)
    };
  }, [dependencies]);
  return (
    <div>
      {/* JSX content */}
    </div>
  );
};
```

Used in functional components for handling side effects

useMemo Hook

```
import React, { useState, useMemo } from 'react';
const MemoizedValue = () => {
  const [count, setCount] = useState(0);
  const squaredValue = useMemo(() => {
    return count * count;
  }, [count]);

  return (
    <div>
      <p>Count: {count}</p>
      <p>Squared Value: {squaredValue}</p>
    </div>
  );
};
```

Memoizing values for optimization

useRef Hook

```
import React, { useRef, useEffect } from 'react';
const InputWithFocus = () => {
  const inputRef = useRef();
  useEffect(() => {
    inputRef.current.focus();
  }, []);

  return <input ref={inputRef} />;
};
```

Accessing and interacting with DOM elements

Custom Hooks

```
import React, { useState, useEffect } from 'react';
const useCustomHook = (initialValue) => {
  const [value, setValue] = useState(initialValue);
  useEffect(() => {
    // Custom logic or side effects
    console.log('Value changed:', value);
  }, [value]);

  return [value, setValue];
};

const CustomHookExample = () => {
  const [customValue, setCustomValue] = useCustomHook(0);
  return (
    <div>
      <p>Custom Value: {customValue}</p>
      <button onClick={() => setCustomValue(customValue + 1)}>Increment</button>
    </div>
  );
};
```

Creating reusable custom hooks

04

Event Handling

An **event** is an occurrence triggered by user interactions, such as clicks, keyboard input, or mouse movements.



React enables handling user interactions, such as clicks or input changes, through event handling mechanisms.

Synthetic Events

Cross-browser event system that normalizes the event handling across different browsers.

```
<button onClick={handleClick}>Click Me</button>

const handleClick = (event) => {
  // Access event properties (e.g., event.target)
};
```

Example

Event Binding

Event binding in React involves establishing a connection between an event (such as a click or input change) and a function that will be executed when the event occurs.

```
const EventComponent = () => {
  const Press = () => {
    // Click logic
  };

  return <button onClick={Press}>Click Me</button>;
};
```

In Function component

```
class MyComponent extends React.Component {
  constructor() {
    super();
    this.handleClick = this.handleClick.bind(this);
  }

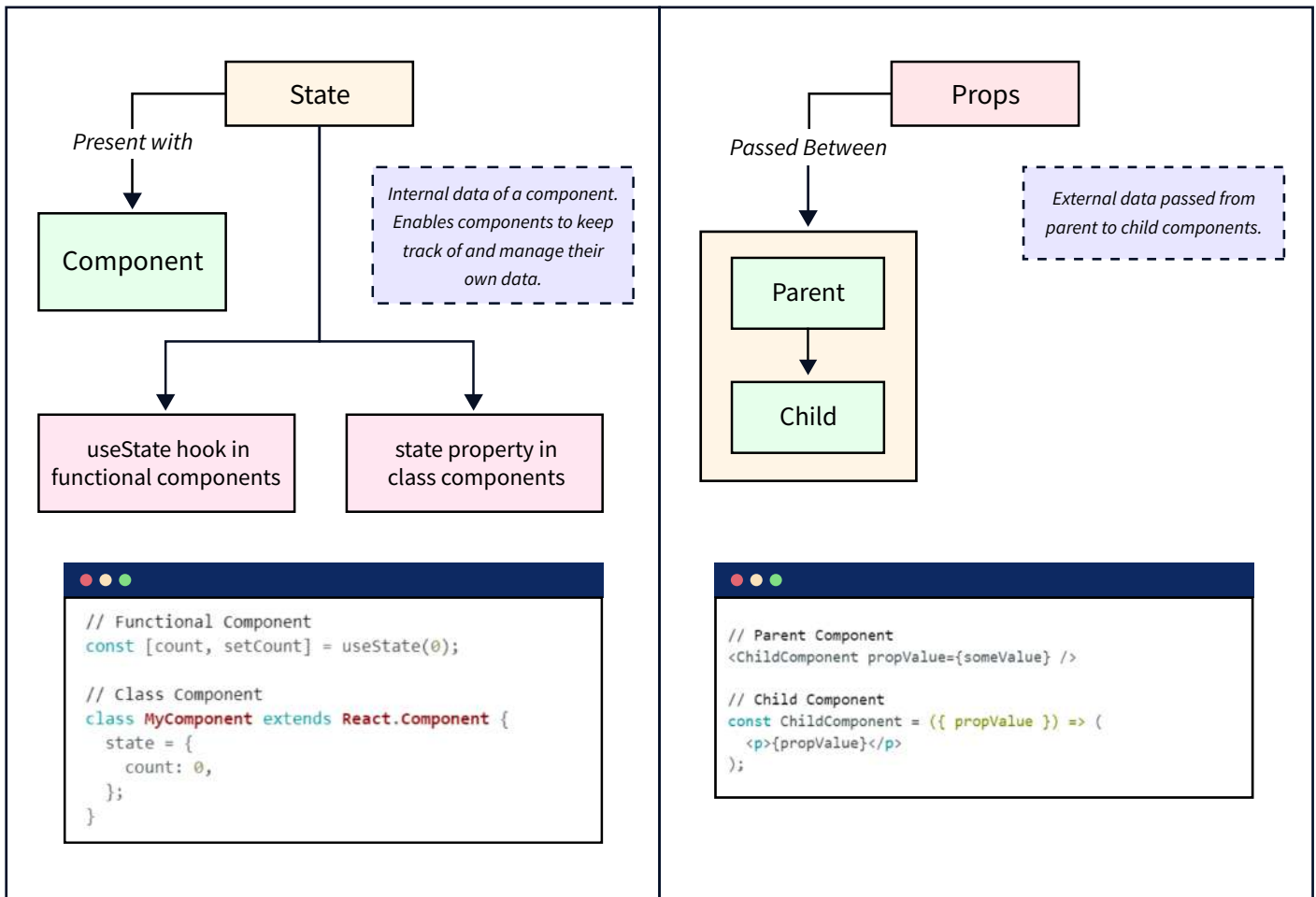
  handleClick() {
    // Handle click
  }

  render() {
    return <button onClick={this.handleClick}>Click Me</button>;
  }
}
```

In Class component

05

Props and State



06

React Router

A library for routing in React applications, enabling navigation and rendering components based on the URL.

Steps to use React Router

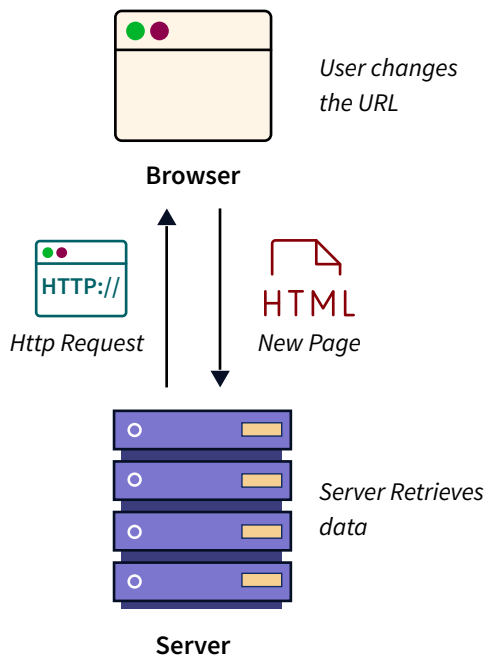
Install react-router-dom using npm

Import BrowserRouter from React router

Use the Link component for navigation.

Add the exact prop to a Route for an exact path match.

Use the Route component to associate a path with a React component.



```
import { BrowserRouter as Router, Route, Link } from 'react-router-dom';
const Home = () => <h2>Home</h2>;
const About = () => <h2>About</h2>;
const App = () => {
  return (
    <Router>
      <div>
        <nav>
          <ul>
            <li>
              <Link to="/">Home</Link>
            </li>
            <li>
              <Link to="/about">About</Link>
            </li>
          </ul>
        </nav>
        <hr />
        <Route path="/" exact component={Home} />
        <Route path="/about" component={About} />
      </div>
    </Router>
  );
};
```

Example

Types of Routing

Dynamic Routing

Routes with route parameters.

Nested Routing

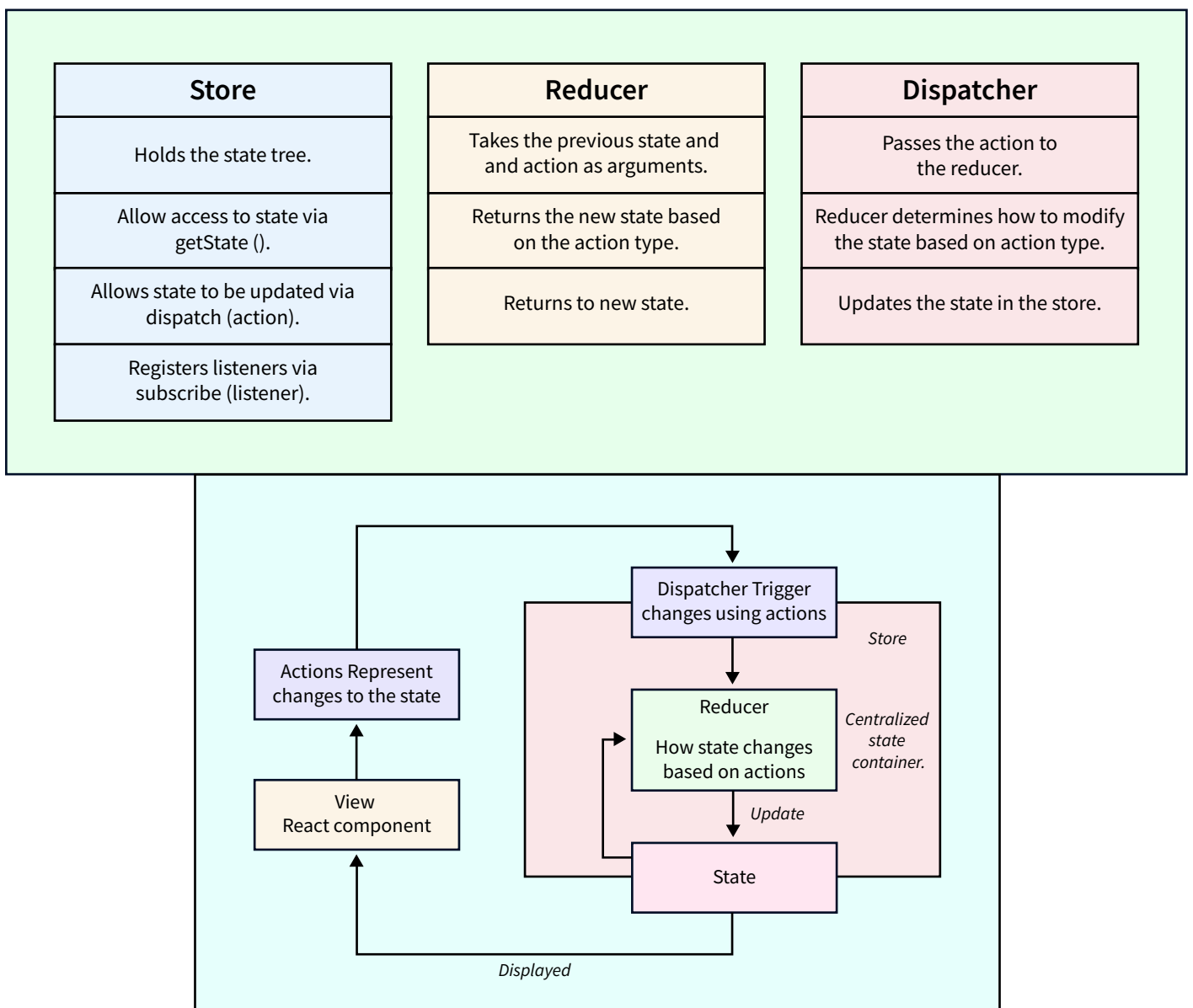
Nest routes within components to create complex layouts.

07

React Redux



Manages the application state in a centralized store, making state changes more predictable.



Examples

```
// Counter
import React from 'react';
import { connect } from 'react-redux';

const Counter = ({ count, increment, decrement }) => {
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
    </div>
  );
};

const mapStateToProps = (state) => ({
  count: state.count,
});

const mapDispatchToProps = (dispatch) => ({
  increment: () => dispatch({ type: 'INCREMENT' }),
  decrement: () => dispatch({ type: 'DECREMENT' }),
});

export default connect(mapStateToProps, mapDispatchToProps)(Counter);
```

```
// reducer.js
const initialState = {
  count: 0,
};

const reducer = (state = initialState, action) => {
  switch (action.type) {
    case 'INCREMENT':
      return { ...state, count: state.count + 1 };
    case 'DECREMENT':
      return { ...state, count: state.count - 1 };
    default:
      return state;
  }
};
```

```
// store.js
import { createStore } from 'redux';
import reducer from './reducer';

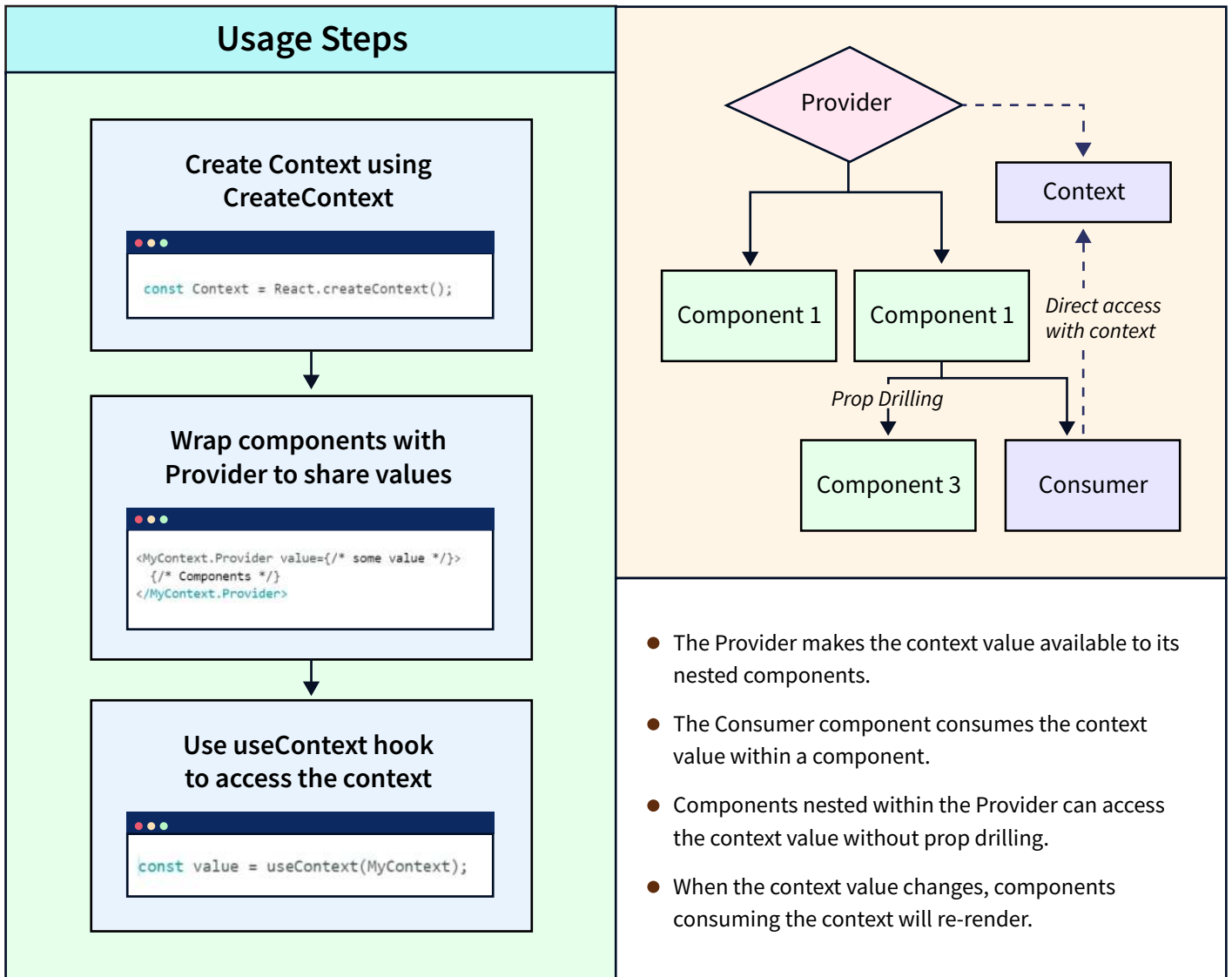
const store = createStore(reducer);

export default store;
```

08

Context API

Way to share values between components without passing props explicitly



09

Conditional Rendering in React

Conditionally render components based on a condition using standard JavaScript if statements.

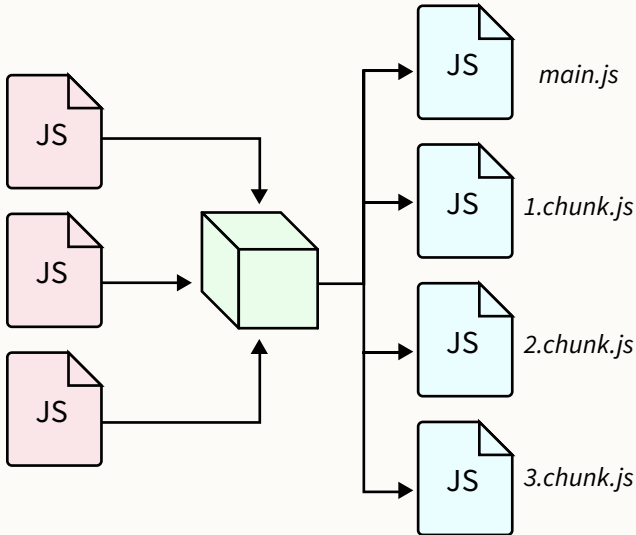


Advantage

Allows components to adapt dynamically based on changing states or props.

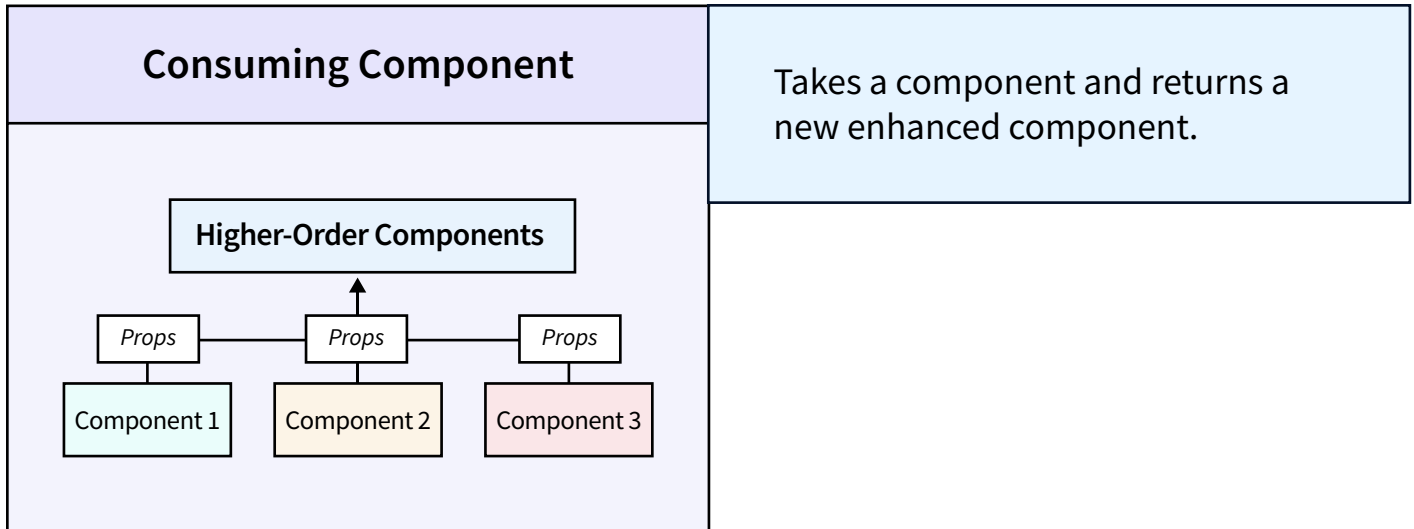
Using Ternary Operators (condition) ? expression1 : expression2	Using Logical Operators(&&)
 Renders Component if condition is true and render AnotherComponent if condition is false	 Renders Component only if the condition is true or renders none.

10 Code Splitting

Example	Dynamically load parts of your application, improving performance.
	

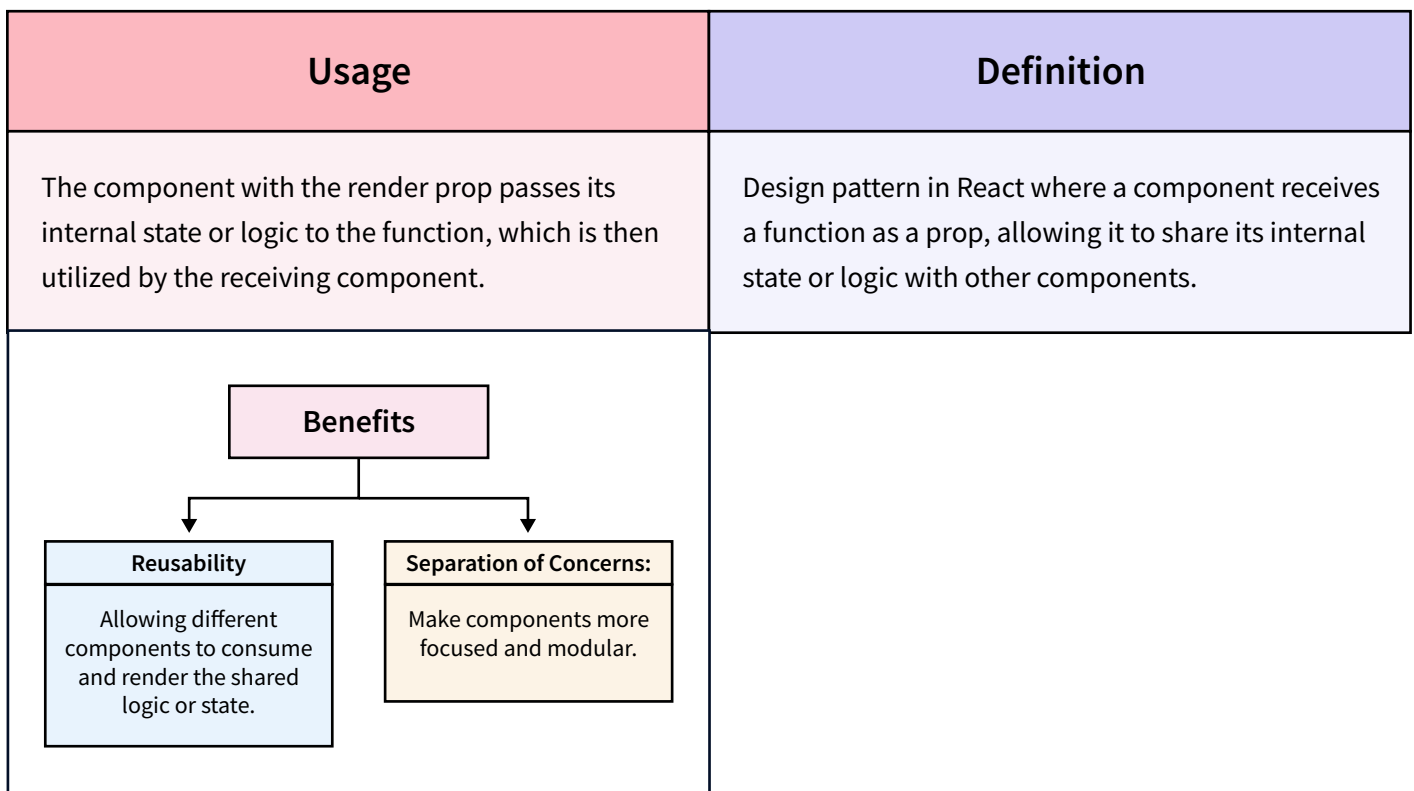
11

Higher Order Component



12

Render Props



Examples

```
// Component with Render Prop
const MouseTracker = ({ render }) => {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  const handleMouseMove = (event) => {
    setPosition({ x: event.clientX, y: event.clientY });
  };

  return <div onMouseMove={handleMouseMove}>{render(position)}</div>;
};

// Receiving Component
const DisplayMousePosition = ({ position }) => {
  return <p>Mouse position: {position.x}, {position.y}</p>;
};

// Usage
<MouseTracker render={position => <DisplayMousePosition
  position={position} /> } />;
```

- 'MouseTracker' component tracks mouse position and passes it to a render function through the 'render' prop.
- 'DisplayMousePosition' component receives the mouse position via the 'position' prop and renders it within the UI, showcasing the reusable nature of the Render Props pattern.

SCALER TOPICS

Unlock your potential in software development with
FREE COURSES from **SCALER TOPICS**!

Register now and take the first step towards your future Success!



PRATEEK NARANG

C++ for Beginners

 5.9k enrolled  **Free**



TARUN LUTHRA

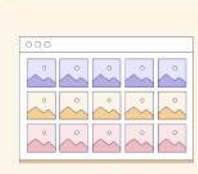
Java for Beginners

 6.8k enrolled  **Free**

That's not it. Explore 20+ Courses by clicking below

[Explore Other Courses](#)

Practice **CHALLENGES**
and become 1% better everyday



CIFAR-10 Image Classification Using PyTorch
Article

 No. Of Questions : 3

[Go to Challenge >](#)



How to Build a Snake Game in JavaScript?
Article

 No. Of Questions : 3

[Go to Challenge >](#)

[Explore Other Challenges](#)